

ATIVIDADE PRÁTICA EM DUPLA

Chat de conversa com sockets

Partam dos quatro arquivos fornecidos. Façam apenas pequenas alterações e testem uma etapa antes de avançar.

Arquivos: 01_texto_bytes.py, 02_servidor_tcp.py, 03_cliente_tcp.py e protocolo_tcp.py.

1 Conectar os dois computadores à mesma rede

- Conectem os dois computadores ao mesmo hotspot.
- No computador que será o servidor, executem ipconfig.
- Localizem o endereço IPv4 da interface Wi-Fi e informem esse IP ao colega.

No servidor

```
HOST = "0.0.0.0"  
PORTA = 9000
```

No cliente

```
HOST = "IP_DO_SERVIDOR"  
PORTA = 9000
```

Atenção: não usem 127.0.0.1 no cliente. Esse endereço aponta para o próprio computador.

2 Executar os arquivos sem mudar a lógica

- Servidor: execute py 02_servidor_tcp.py.
- Cliente: execute py 03_cliente_tcp.py.
- Envie uma mensagem curta. O servidor deverá devolver ECO: mensagem.
- Testem uma frase com espaços e outra com acentos.

Verificação: só avancem quando o eco funcionar entre os dois computadores.

3 Transformar o eco em uma conversa por turnos

No servidor, substituam a resposta automática por uma resposta digitada pelo aluno servidor.

```
print("Cliente:", mensagem)
resposta = input("Resposta: ")
enviar_mensagem(conexao, resposta)
```

- O cliente digita e envia uma mensagem.
- O servidor lê a mensagem, digita uma resposta e envia.
- Repitam o processo pelo menos três vezes.

Funcionamento: neste momento, o chat funciona por turnos: cliente fala e servidor responde.

4 Adicionar um delimitador simples

Use `\n` para marcar o final de cada mensagem. Alterem somente o arquivo `protocolo_tcp.py`.

Envio

```
dados = (mensagem + "\n").encode("utf-8")
socket_conexao.sendall(dados)
```

Recebimento

```
dados = socket_conexao.recv(1024)

if not dados:
    return None

return dados.decode("utf-8").rstrip("\n")
```

- Executem novamente servidor e cliente.
- Testem três mensagens e confirmem que `\n` não aparece na tela.

Limite desta versão: esta é uma primeira solução didática. Como o TCP é um fluxo de bytes, uma versão mais completa precisaria guardar dados em um buffer até encontrar o delimitador.

5 Escolher um miniprotocolo

Escolham uma opção. Cliente e servidor devem usar exatamente o mesmo formato.

Opção	Mensagem normal	Pedir horário	Encerrar
A - comandos	/msg Olá	/hora	/sair
B - tipo e conteúdo	MSG Olá	HORA	SAIR
C - códigos	1 Olá	2	9

Regra: o protocolo escolhido precisa reconhecer somente três ações: mensagem, horário e encerramento.

Exemplo de decisão simples para a opção A

```
if mensagem.startswith("/msg "):
    print("Cliente:", mensagem[5:])
elif mensagem == "/hora":
    print("Cliente pediu o horário")
elif mensagem == "/sair":
    print("Cliente encerrou a conversa")
else:
    print("Comando desconhecido")
```

- Adaptem a decisão ao formato escolhido pela dupla.
- Para o comando de horário, o servidor pode responder com um texto fixo ou usar `datetime.now().strftime("%H:%M")`.
- Ao receber o comando de saída, encerrem o laço com `break`.

6 Testar o protocolo

- Envie duas mensagens normais.
- Peça o horário.
- Envie um comando inexistente.
- Encerrem usando o comando definido no protocolo.

Saída: usem apenas `print()` para mostrar mensagens, respostas e erros. Não criem arquivos de log.

7 Criar uma regra simples de alerta

Escolham e implementem uma regra no servidor.

Regra	Exemplo de saída
Mensagem com mais de 100 caracteres	[ALERTA] Mensagem muito grande
Tipo ou comando desconhecido	[ALERTA] Comando desconhecido
Palavra suspeita escolhida pela dupla	[ALERTA] Palavra suspeita detectada
Três mensagens inválidas	[ALERTA] Muitas mensagens inválidas

Exemplo com limite de tamanho

```
if len(mensagem) > 100:
    print("[ALERTA] Mensagem muito grande")
```

Teste final

- Uma mensagem válida aparece normalmente.
- O pedido de horário recebe uma resposta.
- A condição escolhida produz um alerta no servidor.
- O servidor continua funcionando depois do alerta.
- O comando de saída encerra a conversa.

Atividade concluída

- A conexão funciona entre dois computadores.
- O eco foi transformado em conversa por turnos.
- As mensagens usam \n como delimitador.
- A dupla implementou um dos três miniprotocolos.
- Uma regra de alerta foi testada.

Troca de papéis: quem executou o cliente passa a executar o servidor.